

```

# Establecer semilla aleatoria para reproducibilidad
# set.seed(123) # Descomentar para pruebas fijas, comentar para aleatoriedad total en generación
set.seed(sample(1:100000, 1))

# --- Aleatorización de Parámetros ---

# 1. Coeficiente inicial (cantidad inicial del organismo)
coeficientes_posibles <- c(20, 25, 30, 40, 50, 60, 75, 80, 100, 120, 150, 200)
coef_inicial <- sample(coeficientes_posibles, 1)

# 2. Base de la función exponencial
bases_posibles <- c(1.5, 2, 2.2, 2.5, 3) # Bases mayores a 1 para crecimiento
base_exponencial <- sample(bases_posibles, 1)

# 3. Factor multiplicador en el exponente del tiempo
factores_tiempo_posibles <- c(0.2, 0.25, 0.4, 0.5, 0.75, 0.8, 1, 1.2, 1.5)
factor_tiempo <- sample(factores_tiempo_posibles, 1)

# 4. Tipo de organismo
organismos_plural <- c("ranas", "peces", "insectos", "aves", "conejos", "bacterias", "algas", "hongos",
organismos_singular <- c("rana", "pez", "insecto", "ave", "conejo", "bacteria", "alga", "hongo", "planta")
idx_organismo <- sample(1:length(organismos_plural), 1)
organismo_plural <- organismos_plural[idx_organismo]
organismo_singular <- organismos_singular[idx_organismo]

# 5. Unidad de tiempo
unidades_tiempo_plural <- c("semanas", "días", "meses", "horas", "años", "minutos")
unidades_tiempo_singular <- c("semana", "día", "mes", "hora", "año", "minuto")
idx_unidad <- sample(1:length(unidades_tiempo_plural), 1)
unidad_tiempo_plural <- unidades_tiempo_plural[idx_unidad]
unidad_tiempo_singular <- unidades_tiempo_singular[idx_unidad]

# 6. Verbo para la acción de reproducción/crecimiento
verbos_accion <- c("se reproduce", "crece", "aumenta su población", "se multiplica", "evoluciona su canal")
verbo_accion <- sample(verbos_accion, 1)

# 7. Contexto del estudio/observación
contextos_estudio <- c("un estudio científico", "una observación de campo", "un experimento de laboratorio")
contexto_estudio <- sample(contextos_estudio, 1)

# 8. Variable que representa la cantidad (r, P, N, Q)
simbolo_cantidad <- sample(c("r", "P", "N", "Q", "C"), 1)

# 9. Variable que representa el tiempo (t, x, k)
simbolo_tiempo <- sample(c("t", "x", "k"), 1)

# 10. Adjetivo para la especie
adjetivos_especie <- c("cierta", "una determinada", "una particular", "una nueva", "una exótica")
adjetivo_especie <- sample(adjetivos_especie, 1)

# --- Formulación de la función ---
#  $r(t) = \text{coef\_inicial} * (\text{base\_exponencial})^{(\text{factor\_tiempo} * t)}$ 
# Usamos glue para formatear la función con los símbolos aleatorios

```

```

funcion_texto <- glue::glue("{simbolo_cantidad}({simbolo_tiempo}) = {coef_inicial} \\cdot ({base_exponencial})^{{factor_tiempo}}")
# Para mostrar sin el cdot y con parentesis si la base es decimal
funcion_display <- glue::glue("{simbolo_cantidad}({simbolo_tiempo}) = {coef_inicial}({base_exponencial})^{{factor_tiempo}}")

# --- Cálculo de la Solución ---
# La cantidad inicial es cuando el tiempo (simbolo_tiempo) es 0.
# {simbolo_cantidad}(0) = coef_inicial * (base_exponencial)^(factor_tiempo * 0)
# {simbolo_cantidad}(0) = coef_inicial * (base_exponencial)^0
# {simbolo_cantidad}(0) = coef_inicial * 1
# {simbolo_cantidad}(0) = coef_inicial
respuesta_correcta <- coef_inicial

# --- Generación de Opciones Incorrectas (Distractores) ---
opciones <- numeric(4)
opciones[1] <- respuesta_correcta

# Distractor 1: coef_inicial * base_exponencial
distractor1 <- round(coef_inicial * base_exponencial)
if (distractor1 == respuesta_correcta) {
  distractor1 <- round(coef_inicial * base_exponencial + sample(1:5,1)) # Pequeña variación si coincide
}

# Distractor 2: coef_inicial / base_exponencial (si es razonable y entero) o coef_inicial / 2
if (coef_inicial %% base_exponencial == 0 & base_exponencial != 1) {
  distractor2 <- round(coef_inicial / base_exponencial)
} else {
  distractor2 <- round(coef_inicial / 2)
}
if (distractor2 == respuesta_correcta || distractor2 == distractor1) {
  distractor2 <- round(coef_inicial / (base_exponencial + 0.5)) # Variación
  if (distractor2 <= 0) distractor2 <- round(coef_inicial * 0.75)
}

# Distractor 3: base_exponencial o factor_tiempo (si son enteros y pequeños) o un valor aleatorio pequeño
if (base_exponencial %% 1 == 0 && base_exponencial > 1 && base_exponencial < 10) {
  distractor3 <- base_exponencial
} else if (factor_tiempo %% 1 == 0 && factor_tiempo > 1 && factor_tiempo < 10) {
  distractor3 <- factor_tiempo
} else {
  distractor3 <- sample(c(2,3,4,5,10),1)
}

# Asegurar que todos los distractores sean únicos y diferentes de la respuesta correcta
# y que sean positivos
distractores_potenciales <- c(distractor1, distractor2, distractor3)
distractores_unicos <- unique(distractores_potenciales[distractores_potenciales > 0 & distractores_potenciales != respuesta_correcta])

# Si no tenemos suficientes distractores únicos, generamos más
iter_seguridad = 0
while(length(distractores_unicos) < 3 && iter_seguridad < 20) {
  nuevos_distractores <- c(

```

```

    round(coef_inicial * (base_exponencial + runif(1, -0.5, 0.5))),
    round(coef_inicial * runif(1, 0.25, 2.5)),
    round(coef_inicial + sample(c(-1,1),1) * sample(5:15,1)),
    sample(c(1,2,3,4,5,10,15,20),1) * sample(c(1,2),1)
  )
  distractores_potenciales <- c(distractores_unicos, nuevos_distractores)
  distractores_unicos <- unique(distractores_potenciales[distractores_potenciales > 0 & distractores_
distractores_unicos <- distractores_unicos[!is.na(distractores_unicos)] # Eliminar NA
  iter_seguridad = iter_seguridad + 1
}

# Si después de los intentos aún no hay suficientes, usar valores fijos relativos a la respuesta
if (length(distractores_unicos) < 3) {
  distractores_fijos = c(round(respuesta_correcta * 1.5), round(respuesta_correcta * 0.5), round(respuesta_correcta * 0.5))
  distractores_unicos = unique(c(distractores_unicos, distractores_fijos))
  distractores_unicos = distractores_unicos[distractores_unicos > 0 & distractores_unicos != respuesta_correcta]
}

# Tomar los primeros 3 distractores únicos
opciones[2:4] <- sample(distractores_unicos, min(3, length(distractores_unicos)))[1:min(3, length(distractores_unicos))]

# Si faltan opciones, rellenar con valores genéricos para asegurar 4 opciones
# Esto es un fallback, idealmente los distractores generados son suficientes
if (any(is.na(opciones[2:4])) || length(unique(opciones)) < 4) {
  opciones_faltantes_idx <- which(is.na(opciones) | duplicated(opciones, fromLast = TRUE) | duplicated(opciones, fromFirst = TRUE))
  opciones_faltantes_idx = opciones_faltantes_idx[!is.na(opciones_faltantes_idx)]

  # Asegurar que las opciones sean enteras
  opciones <- round(opciones)

  # Generar valores de relleno si es necesario
  rellenos_base <- c(respuesta_correcta * 2,
                    max(1, round(respuesta_correcta / 2)),
                    respuesta_correcta + base_exponencial,
                    max(1, respuesta_correcta - base_exponencial),
                    base_exponencial,
                    coef_inicial + 5,
                    coef_inicial - 5)
  rellenos_validos <- unique(round(rellenos_base[rellenos_base > 0]))
  rellenos_disponibles <- setdiff(rellenos_validos, opciones)

  idx_reemplazo = 2 # Empezar a reemplazar desde la segunda opción si es necesario
  while(length(unique(opciones)) < 4 && idx_reemplazo <= 4 && length(rellenos_disponibles) > 0) {
    if (opciones[idx_reemplazo] == respuesta_correcta || sum(opciones == opciones[idx_reemplazo]) > 1) {
      opciones[idx_reemplazo] = rellenos_disponibles[1]
      rellenos_disponibles = rellenos_disponibles[-1]
    }
    idx_reemplazo = idx_reemplazo + 1
    opciones <- round(opciones) # Asegurar enteros
  }

  # Si aún no hay 4 opciones únicas, forzar valores simples
  if (length(unique(opciones)) < 4) {
    opciones_unicas_actuales <- unique(opciones[!is.na(opciones)])
    complemento <- c(1,2,3,4,5,10,15,20,25,30,40,60,70,80,90) # Valores de relleno
  }
}

```

```

complemento_disponible <- setdiff(complemento, opciones_unicas_actuales)

for(i in 1:4){
  if(sum(opciones == opciones[i]) > 1 || is.na(opciones[i])){ # Si está duplicada o NA
    if(length(complemento_disponible) > 0){
      opciones[i] <- complemento_disponible[1]
      complemento_disponible <- complemento_disponible[-1]
    } else {
      opciones[i] <- respuesta_correcta + i*5 # Último recurso
    }
  }
}
}

opciones <- round(opciones) # Asegurar que todas las opciones sean enteras

# Ordenar las opciones para la presentación (opcional, exshuffle=TRUE se encarga)
# opciones_ordenadas <- sample(opciones) # Mezclar opciones
opciones_ordenadas <- opciones # Mantener el orden para que la solución coincida con el índice 1

# Crear el vector de solución para r-exams (TRUE/FALSE)
solucion_logica <- opciones_ordenadas == respuesta_correcta

```

Question

cierta especie de algas se multiplica según la función $Q(k) = 80 \cdot (2)^{0.5 \cdot k}$, donde k se mide en días y Q es la cantidad de algas.

De acuerdo con la información anterior, la cantidad de algas que había al iniciar un análisis poblacional, es:

Answerlist

- 80 algas.
- 160 algas.
- 40 algas.
- 2 algas.

Solution

Para determinar la cantidad de algas que había al iniciar un análisis poblacional, necesitamos evaluar la función de población $Q(k) = 80(2)^{0.5k}$ en el momento inicial, es decir, cuando el tiempo $k = 0$.

La función dada es: $Q(k) = 80(2)^{0.5k}$

Sustituimos $k = 0$ en la función: $Q(0) = 80(2)^{0.5 \cdot 0}$ $Q(0) = 80(2)^0$

Cualquier número (distinto de cero) elevado a la potencia 0 es igual a 1. Por lo tanto, $(2)^0 = 1$. $Q(0) = 80 \cdot 1$
 $Q(0) = 80$

Así, la cantidad de algas que había al iniciar un análisis poblacional es 80.

Answerlist

- Verdadero
- Falso

- Falso
- Falso

Meta-information

exname: crecimiento_exponencial_valor_inicial extype: schoice exsolution: 1000 exshuffle: TRUE # Mezclar el orden de las opciones de respuesta exsection: Álgebra y Cálculo/Funciones Exponenciales/Modelado de Poblaciones extol: 0.01 # Tolerancia para respuestas numéricas (no aplica a schoice directamente) expoints: 1 # Puntos asignados a esta pregunta